ABSTRACTION

Abstraction is OOPS principle which is used to provide only the necessary details or essential features of an object & hide the implementation of it from the user.

In Java we can achieve Abstraction with the help of
1) Class 2) Interface.

ABSTRACTION WITH THE HELP OF CLASS:-

1) With the help of class we can achieve Abstraction from 0 to 100%.

2) Abstraction class can have two types of Methods w.r.t Abstraction:

a) Concrete Method

b) Abstract Method.

CONCRETE METHOD :

A Method which has method implementation is known as Concrete Method.

SYNTAX:

```
[Modifiers] returntype MethodName([formal Args])
{
  //java statements.
```

Eg:
```
public int test()
{
    System.out.println("from test()");
    return 10;
}
```

ABSTRACT METHOD:

A Method which doesn't have Method implement is known as Abstract Method

SYNTAX:

```
abstract [Access Modifiers] returntype MethodName([Formal Args]);
```

Eg:
```
abstract public void add(int a, int b);
```

With Respect to Abstraction we can classify classes into two types

1) Abstract Class
2) Concrete class

ABSTRACT CLASS:

A class which is prefixed with abstract modifier is known as abstract class.

Syntax:

```
abstract Class P1
{
  // java statements
}
```

When should we make a class Abstract?

1) If we have an Abstract method declared inside my class
2) If we have an Abstract method Inherited but not overriden

IMPORTANT POINTS TO BE REMEMBERED ABOUT ABSTRACT CLASS:

1) Abstract classes have constructors ✓
2) Abstract classes can have both concrete method & Abstract method
3) We can make class Abstract even if it is not having the abstract method.
4) Variable cannot be abstracted
5) We cannot instantiate for abstract class ie we can't create object for Abstract classes.
6) Static members can't be abstracted.
7) private members can't be ~~abstracted~~. prefixed with a abstract modifier.
8) final members cant be abstracted.

CONCRETE CLASS:

A class which is not prefixed with Abstract modifier is known as Concrete class.

A concrete class cannot have any abstract methods.

A concrete class can be instantiated.

CONCRETE IMPLEMENTING CLASS:-

A class inside which we provide implementation to all the abstract methods using Method overriding is known as Concrete Implementing class.

STEPS TO CREATE A CONCRETE IMPLEMENTING CLASS:-

STEP1: Create a sub-class for the Abstract class.

STEP 2: Provide Implementation to all the Methods which is present inside Abstract class.

Eg:
```
abstract class Calci
{
abstract int add (int a, int b);
abstract int sub (int a, int b);
abstract int mul (int a, int b);
abstract int div (int a, int b);
}
```

```
class Calculator extends Calci
{
public int add (int a, int b)
{
 return a+b;
}
public int sub (int a, int b)
{
return a-b;
}
public int mul (int a, int b)
{
 return a*b;
}
public int div (int a, int b)
{
 return a/b;
}
}
 class Driver
 {
  public static void main (String [] args)
  {
   Calculator c1 = new Calculator ();
S.o.pln(c1.add (10, 20));
 " (c1. sub (10, 20));
 " (c1. mul (10, 20));
 " (c1. div (20, 10));
  }
}
```

Eg2 :

```
abstract class Laptop
{
 int id;
 String ram;
 double price;
 Laptop(){ }
 Laptop(int id, String ram, double price)
 {
  this.id= id;
  this.ram= ram;
  this.price= price;
 }
 abstract void displayDetails();
}

class Lap extends Laptop
{
 public void displayDetails()
 {
  System.out.println("ID of laptop:" + id);
  System.out.println("RAM of Laptop:" + ram);
  System.out.println("price of Laptop:" + price);
 }
}

class Driver1
{
 public static void main(String[] args)
 {
  Lap L1= new Lap(101,"4GB", 10000);
  L1.displayDetails();
 }
}
```

//add constructor
Lap(){ }
Lap(int id, String ram, double pr
 super(id, ram, price);
 }

▶ interface :-

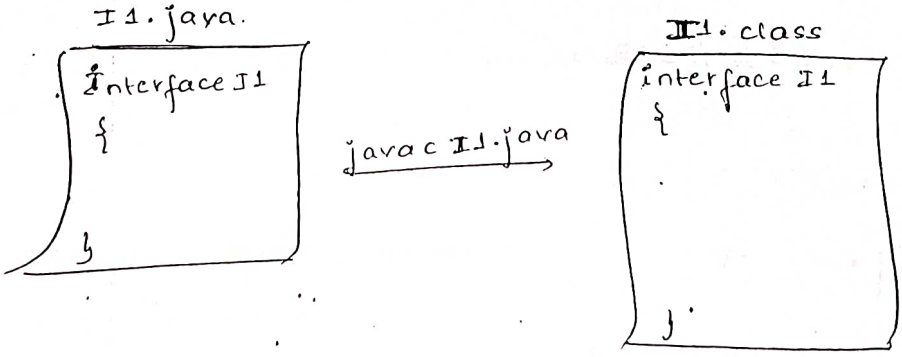
* It is a keyword.
* interface is a user defined non-primitive data type which is used to achieve Multiple Inheritance & 100% Abstraction.

SYNTAX TO CREATE INTERFACE :

```
interface InterfaceName
{
    // java statements
}
```

NOTE :

1) By default interface is an abstract member.
2) We cannot create an Object for interface.
3) We can compile an interface & generate a class file for that.



4) We cannot make an interface either private or protected. It can be either default or public.

5) We cannot make an interface as final member.

Members we can have inside Interface:

Inside interface we can have

1) Static Methods.

Eg:
```
interface I2
{
    public static void main ("String [] args)
    {
        System.out.println ("Hello world");
    }
}
```

O/p:
Hello world.

With the above example we can say two important things.

1) We can have static methods inside Interface.

2) We can execute the interface.

2) ~~Abs~~ public abstract method.

Eg:
```
interface I4
{
    void test(); //CTS
}
```

By default the test method inside interface I4 will be prefixed with public & abstract modifier.

Program to prove test() is prefixed with public modifier

Eg:

```
class A implements I4
{
  void test()
  {
    System.out.println("Implemented inside class A");
  }
}
```

The above program is going to throw compile time error saying "attempting to assign weaker access priveleges. ; was public" which means test() of super member ie interface I4 was a public member.

3) public, static & final variable.

```
interface I6
{
  int i = 10;
  public static void main (String [] args)
  {
    System.out.println (i); //CTS → static variable
    i = 20; //CTE → final variable
  }
}
```

By default variable i will be prefixed with public static & final modifiers.

Members we cannot have inside Interface :

1) Concrete (non-static) methods

2) final & abstract methods

3) Non-static variable

4) static block

5) Non-static block

6) Constructors.

7) private & protected members.

8) Non-final static variable.

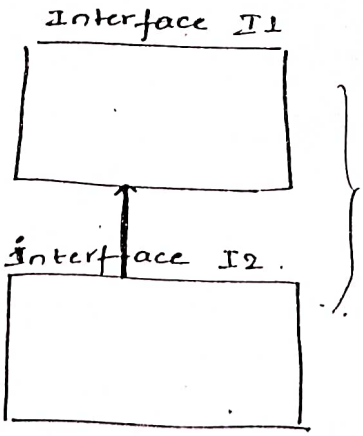
INHERITANCE W.R.T INTERFACES

NOTE :

1) The static methods of interfaces are not Inherited.

2) The Abstract methods & the public static final variables will be inherited.

CASE 1 : Interface Inheriting Interface

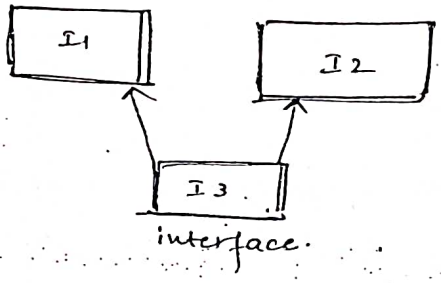


Single level →

```
interface I1
{
}
interface I2 extends I1
{
}
```

An interface can inherit from one or more interfaces. with the help of 'extends' keyword

Eg: Multiple Inheritance

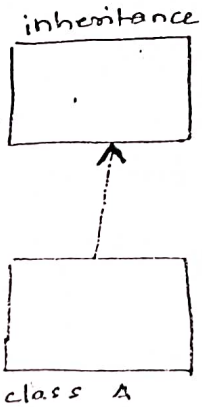


interface.

```
interface I1
{
}
interface I2
{
}
interface I3 extends I1, I2
{
}
```

CASE2 : Class Inheriting from Interface

⇒ A class can inherit from an Interface using "implement keyword.

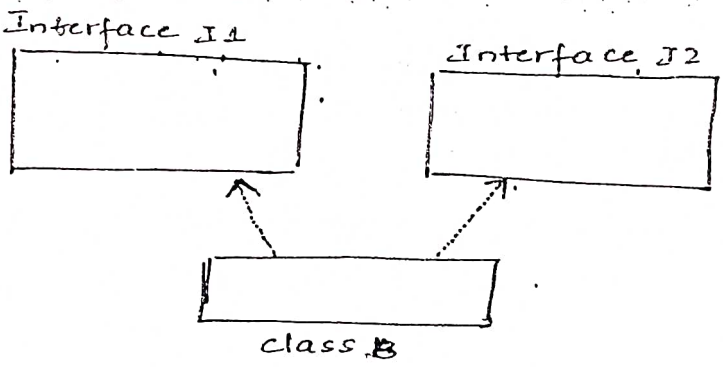


implement is represented using dotted line.

2. interface I1
{
}
class A implements ~~extends~~ I1
{
}

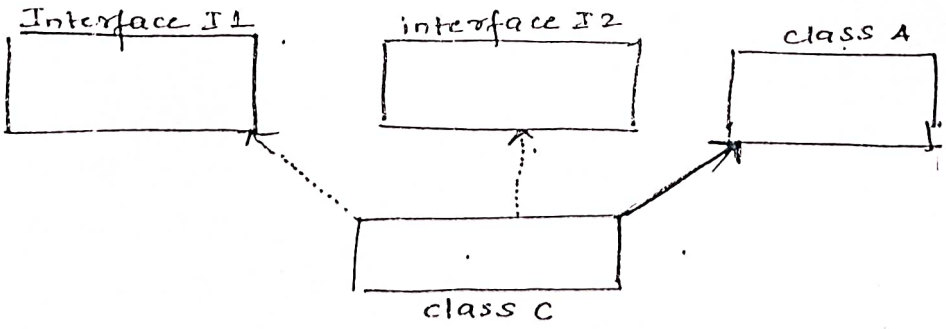
b) A class can inherit from any number of interfaces

Eg:



interface I1
{
}
interface I2
{
}
class B implements ~~extends~~ I1, I2
{
}

3) A class can inherit from one class & any no of interfaces



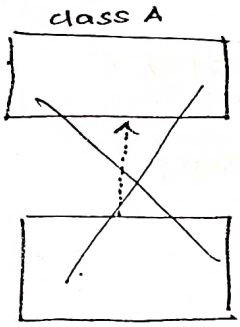
```
interface I1        interface I2        class A
{                   {                   {
}                   }                   }

class C extends A implements I1, I2
{

}
```

NOTE: Whenever a class is inheriting from both the types of members ie a class as well as interfaces we must always make sure we use extends keyword first & then implements, else we get CTE

CASE3: An Interface Inheriting from class



Inheritance I1

In java, it is not possible for an inheritance to inherit from a class.

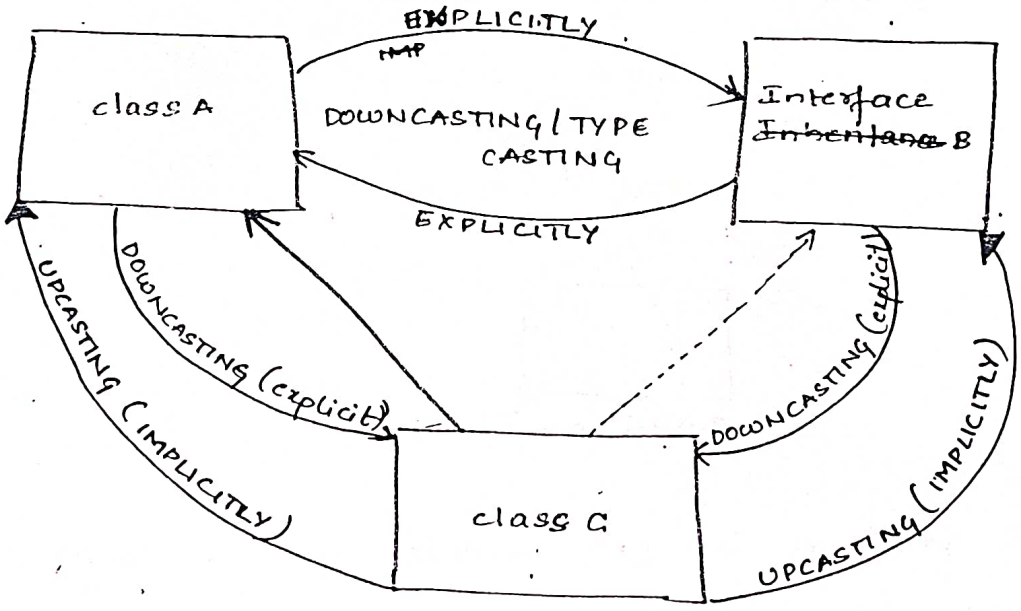
MULTIPLE INHERITANCE

In java, multiple Inheritance is possible with the help of following members

1) Only Interfaces

2) Combination of Class & Interface.

NOTE: Whenever we have multiple inheritance achieved, we can convert one parent type into another parent type. ~~Exp~~ only explicitly.



Eg: class A
{
}
interface B
{
}
class C extends A implements B
{
}

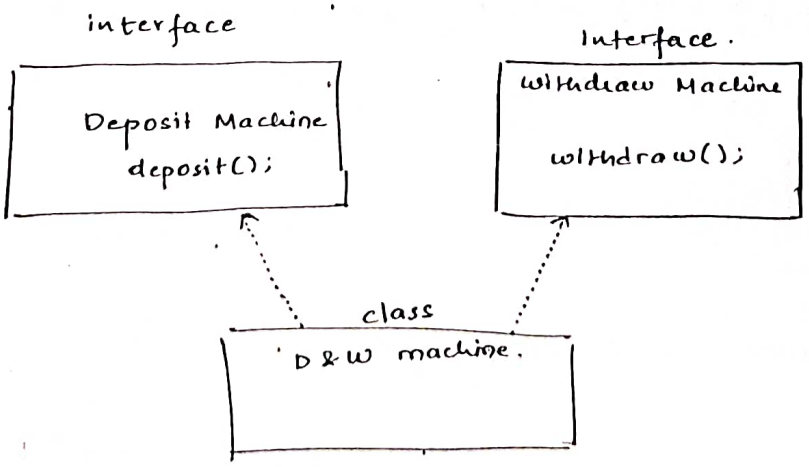
super members being an interface & sub member being a class is also possible for Non-primitive type casting

```
class Driver1
{
 public static void main (String[] args)
 {
    C obj= new C();
    B obj1 = (B) obj ; // UPCASTING
    obj = (C) obj1 ; // DOWNCASTING.
 }
}
```

Conversion of one parent type into another parent type:

```
class Driver2
{
public static void main (String[] args)
 {
  A obj= new A();
  B obj1 = (B) obj;
  obj = (A) obj1;
 }
```

Eg: write program for below Requirement:



```
interface Deposit
{
    int deposit(int amt);
}

interface Withdraw
{
    int withdraw(int amt);
}

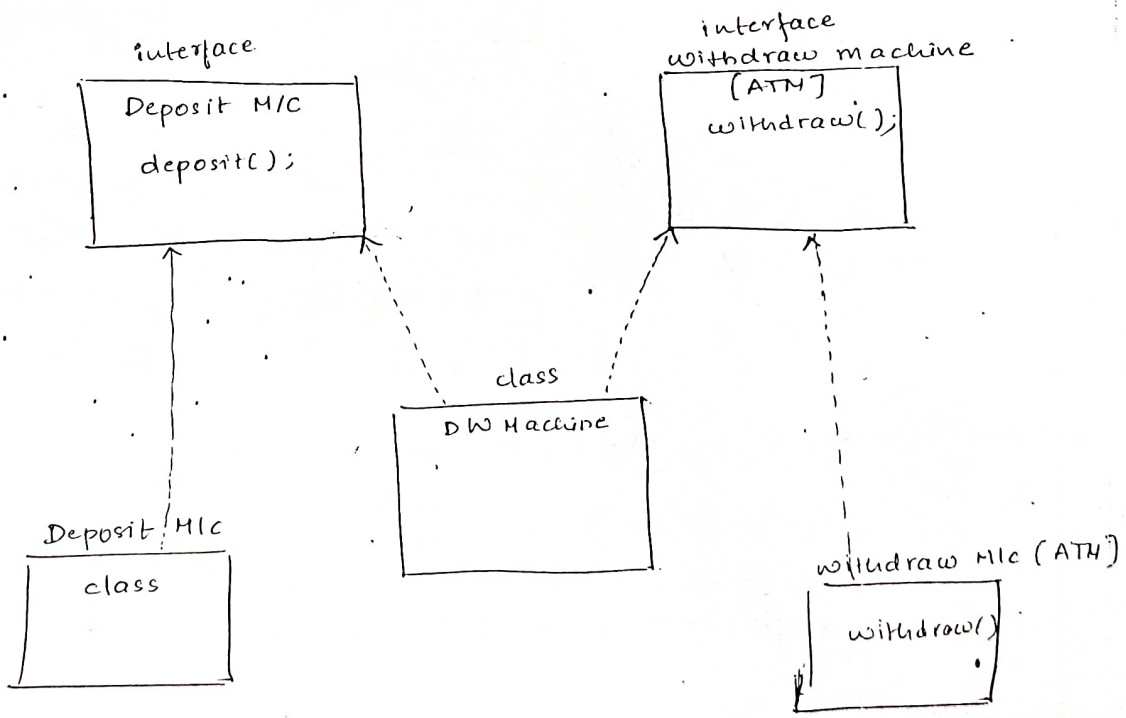
class DepositAndWithdrawMachine implements Deposit, Withdraw
{
    int bal;
    DepositAndWithdrawMachine()
    {
    }
public int deposit(int amt)
    {
        bal = bal + amt;
        System.out.println("Account balance:" + bal);
        return bal;
    }
public int withdraw(int amt)
    {
        if(amt > bal)
            bal -= amt;
        else
            System.out.println("No sufficient balance");
        System.out.println("Account balance:" + bal);
        return bal;
    }
}

class Driver
{
    public static void main(String[] args)
    {
        DepositAndWithdrawMachine D1 = new DepositAndWithdrawMachine();
```

```
D1.deposit(10000);
D1.withdraw(2000);
}
}
```

(OR)

```
class Driver
{
public static void main(String[] args)
{
DepositAndWi
Deposit m1 = new DepositAndWithdrawMachine();
m1.deposit(10000);
((Withdraw) m1).withdraw(2000);
}
}
```



ctrl + shift + O → organise → cmd to do imports / organise

ctrl + d → delete a particular line.

Alt + shift + N → shortcut to open new options.

Short cut to provide suggestions → ctrl + space

command to align the pgm → ctrl + I.

ctrl + ⇒ font increase

ctrl — ⇒ font decrease

Source ⇒ readymade options ⇒ getter, setter, constructor, overriding